Imagine uma situação simples: temos uma lista de ativos financeiros, mas não queremos comprar todos. Precisamos escolher uma quantidade fixa deles. A pergunta é: *quais ativos devemos selecionar, levando em conta o retorno esperado e o risco de combiná-los?*

Vamos acompanhar a resposta desde os preços disponíveis no mercado até os números finais entregues pelo algoritmo. No caminho, explicaremos por que usamos uma função de energia, por que construímos 18 informações para cada operação do circuito, como o Transformer produz os ângulos e como verificamos se esses ângulos ajudam a encontrar uma boa carteira.

As referências originais da literatura aparecem ao longo do texto e ao final. Todo o exemplo com quatro ativos usa números **inventados apenas para ensinar as contas**. Os resultados experimentais reais aparecem somente perto do final, em tabelas separadas.

Comecemos por uma escolha que qualquer pessoa pode entender

Temos quatro ativos, que chamaremos de A, B, C e D, mas só podemos selecionar dois. Essa tarefa é abreviada por $4C2$, isto é, *quatro ativos, escolher dois*.

Uma carteira pode ser representada por uma sequência de zeros e uns:

$$x = (x_A, x_B, x_C, x_D).$$

O número 1 significa "selecionado" e o número 0, "não selecionado". Se escolhermos A e C, escrevemos

$$x = (1, 0, 1, 0), \qquad x_A + x_B + x_C + x_D = 2.$$

A última igualdade é a nossa **restrição de cardinalidade**: precisamos ter exatamente dois números iguais a 1.

Carteira	Vetor de seleção
A+B	$(1, 1, 0, 0)$
A+C	$(1, 0, 1, 0)$
A+D	$(1, 0, 0, 1)$
B+C	$(0, 1, 1, 0)$
B+D	$(0, 1, 0, 1)$
C+D	$(0, 0, 1, 1)$

Neste exemplo, podemos listar as seis possibilidades. Em geral, o número de escolhas possíveis é

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

O símbolo ! indica o fatorial: $4! = 4 \cdot 3 \cdot 2 \cdot 1$. Para quatro ativos e duas escolhas,

$$\binom{4}{2} = \frac{4 \cdot 3 \cdot 2 \cdot 1}{(2 \cdot 1)(2 \cdot 1)} = 6.$$

Já em $10C4$, há 210 carteiras. Em $40C20$, são mais de 137 bilhões. Precisamos de uma forma de *procurar* soluções boas sem ter de examinar todas as combinações.

De onde vêm os números financeiros?

1. Preço é o ponto de partida; retorno mede a variação

Suponha que um ativo valia R$ 100 ontem e R$ 102 hoje. Seu retorno diário simples será

$$r = \frac{102}{100} - 1 = 0{,}02 = 2\%.$$

Em geral, para o ativo i, no pregão t, usamos

$$\boxed{r_{t,i} = \frac{P_{t,i}}{P_{t-1,i}} - 1}.$$

Aqui, $P_{t,i}$ é o preço ajustado do ativo i no dia t, e $P_{t-1,i}$ é o preço ajustado no pregão anterior. O ajuste é importante para tornar as séries coerentes com eventos como desdobramentos.

Uma observação isolada não é suficiente. Por isso o algoritmo analisa uma janela de $w = 60$ ou 90 pregões e calcula a média dos retornos de cada ativo:

$$\overline{r}_i = \frac{1}{w} \sum_{t=1}^{w} r_{t,i}.$$

2. Retorno médio e covariância são duas informações diferentes

O vetor μ descreve os retornos médios anualizados:

$$\boxed{\mu_i = 252\,\overline{r}_i}.$$

O fator 252 é uma aproximação usual do número de pregões em um ano. Aqui a anualização é *linear*, não uma previsão garantida de retorno futuro.

A matriz Σ descreve como os retornos variam conjuntamente:

$$\boxed{\Sigma_{ij} = 252\ \mathrm{Cov}(r_i, r_j)}.$$

Se $i = j$, temos Σ_{ii}, a **variância** do ativo i. Se $i \neq j$, temos Σ_{ij}, a **covariância** entre os ativos i e j.

A variância indica quanto o retorno de um ativo oscila. A covariância indica se dois ativos tendem a subir e cair juntos, ou a se mover de forma menos sincronizada. Não diz, por si só, se a combinação é boa ou ruim: precisamos analisar o problema como um todo.

Para tornar as contas visíveis, adotaremos os valores ilustrativos

$$\mu = \begin{pmatrix} 0{,}10 \\ 0{,}08 \\ 0{,}12 \\ 0{,}06 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 0{,}040 & 0{,}010 & 0{,}008 & 0{,}004 \\ 0{,}010 & 0{,}030 & 0{,}006 & 0{,}003 \\ 0{,}008 & 0{,}006 & 0{,}050 & 0{,}007 \\ 0{,}004 & 0{,}003 & 0{,}007 & 0{,}020 \end{pmatrix}.$$

As posições A, B, C e D correspondem, respectivamente, aos índices $0, 1, 2, 3$ no programa.

Como dizemos matematicamente se uma carteira é boa?

Precisamos transformar a escolha dos ativos em **um número que possamos comparar**. A função utilizada no experimento é

$$\boxed{E_H(x) = \frac{1}{2}x^\mathsf{T}\Sigma x - \mu^\mathsf{T}x}.$$

Vamos desmontar essa fórmula.

Primeiro termo: risco de combinação.

$$x^\mathsf{T}\Sigma x = \sum_{i=1}^{n}\sum_{j=1}^{n} x_i\Sigma_{ij}x_j.$$

O termo inclui as variâncias dos ativos escolhidos e as covariâncias entre eles. O fator $1/2$ é o peso fixado nesta função objetivo para o termo de risco, determinando sua importância relativa em comparação com o retorno. Não representa uma lei da natureza nem é o único peso financeiro possível.

Segundo termo: retorno esperado somado.

$$\mu^\mathsf{T}x = \sum_{i=1}^{n}\mu_i x_i.$$

Subtraímos esse termo porque queremos *minimizar* E_H: um retorno esperado maior tende a tornar a energia menor, enquanto um risco maior tende a aumentá-la.

Cuidado: Como x_i vale 0 ou 1, esta função é um **escore de seleção de ativos**. Ela não normaliza automaticamente os investimentos para pesos $1/k$; por isso seu valor não deve ser confundido com o retorno ou a variância efetiva de uma carteira investida com pesos definidos.

Calculemos a carteira A+C, sem pular nenhuma operação

Neste caso,

$$x = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}.$$

O retorno esperado somado é

$$\mu^\mathsf{T}x = 0{,}10(1) + 0{,}08(0) + 0{,}12(1) + 0{,}06(0) = 0{,}22.$$

O termo de risco inclui apenas as posições A e C:

$$x^\mathsf{T}\Sigma x = \Sigma_{AA} + \Sigma_{AC} + \Sigma_{CA} + \Sigma_{CC} = 0{,}040 + 0{,}008 + 0{,}008 + 0{,}050 = 0{,}106.$$

Logo,

$$E_H(A+C) = \frac{1}{2}(0{,}106) - 0{,}22 = 0{,}053 - 0{,}22 = \boxed{-0{,}167}.$$

Repetindo as contas para as seis carteiras:

Carteira	Energia calculada
A+B	−0,135
A+C	**−0,167**
A+D	−0,126
B+C	−0,154
B+D	−0,112
C+D	−0,138

Neste pequeno exemplo, verificamos por enumeração que A+C tem a menor energia. **No treinamento real o Transformer não recebe essa tabela nem a identidade da carteira ótima.** Só sabemos calcular $E_H(x)$ quando uma carteira candidata aparece.

O símbolo H indica o problema de energia associado aos dados financeiros. Podemos formalizá-lo como um operador diagonal,

$$H = \sum_{x \in \{0,1\}^n} E_H(x)|x\rangle\langle x|,$$

mas não precisamos construir essa matriz enorme para avaliar uma carteira binária.

Por que criar 32 carteiras de referência antes do Transformer?

É útil dar à rede uma noção de *escala*: uma energia de −0,2 pode ser excelente em um problema e ruim em outro. Para isso, sorteamos 32 carteiras com exatamente k ativos, calculamos suas energias e extraímos três números.

Para tornar a ideia concreta, suponha que, no nosso exemplo de apenas seis carteiras possíveis, os 32 sorteios tenham as seguintes contagens. Repetições são permitidas, como também pode acontecer na amostragem do programa.

Carteira	Energia	Vezes entre 32 sorteios
A+B	−0,135	5
A+C	−0,167	5
A+D	−0,126	6
B+C	−0,154	5
B+D	−0,112	5
C+D	−0,138	6
Total		32

Média do contexto: somamos as 32 energias, contando suas repetições, e dividimos por 32:

$$\bar{E}_c = \frac{5(-0{,}135) + 5(-0{,}167) + 6(-0{,}126) + 5(-0{,}154) + 5(-0{,}112) + 6(-0{,}138)}{32} = \boxed{-0{,}13825}.$$

Dispersão do contexto: o desvio-padrão mede o espalhamento das energias:

$$s_c = \sqrt{\frac{1}{32}\sum_{a=1}^{32}(E_a - \bar{E}_c)^2} \approx \boxed{0{,}017475}.$$

No código, o desvio é calculado com divisor 32, e não com 31.

Melhor energia do contexto:

$$E_{c,\min} = \min_{1\le a\le 32} E_a = \boxed{-0{,}167}.$$

O mínimo do contexto é apenas a melhor solução observada entre as 32 amostras. Neste exemplo ele coincide com a melhor carteira das seis, mas **isso não é garantido para problemas maiores**.

Agora já temos dados suficientes para montar a entrada da rede.

Por que entram operações quânticas e ângulos?

Em vez de o Transformer escrever diretamente uma lista de ativos, ele recebe a descrição de um circuito parametrizado e devolve números chamados **ângulos**. Esses números controlam rotações e, por consequência, as probabilidades das carteiras produzidas quando o circuito é medido.

Para uma escolha nCk, o circuito usado possui

$$\boxed{g = k(n-k)}$$

ângulos ajustáveis. No exemplo $4C2$, são $g = 2(4-2) = 4$ ângulos. Em $10C4$, são 24; em $12C5$, são 35.

Cada operação, ou *gate*, age sobre posições de qubits determinadas por uma sequência fixa do algoritmo. Chamaremos essas posições de i e l. Dois tipos aparecem no código: CY, uma rotação controlada, e CCY, uma operação com controle adicional. A rede não escolhe livremente onde cada gate aparece: ela corrige os ângulos de uma estrutura predefinida.

A construção inicial fornece um ângulo de referência $\phi_{D,j}$, relacionado à preparação do estado de Dicke. O circuito começa em um *estado binário de cardinalidade fixa*; quando utiliza os ângulos de referência em sua sequência de gates, produz a preparação de Dicke correspondente. Não estamos aplicando duas vezes a preparação.

Para a operação de índices (i, l), o programa calcula primeiro

$$q_{il} = \sqrt{\frac{l-i}{l+1}}, \qquad \phi_{D,j} = \arccos(q_{il}).$$

A segunda igualdade é válida para a coordenada angular interna adotada nos dois tipos de gate desse circuito. Por exemplo, no **primeiro gate do 4C2**, $i = 1$, $l = 3$, $j = 0$, e o tipo é CCY. Logo,

$$q_{1,3} = \sqrt{\frac{3-1}{3+1}} = \sqrt{\frac{2}{4}} = 0{,}707107, \qquad \phi_{D,0} = \arccos(0{,}707107) = \frac{\pi}{4} \approx 0{,}785398.$$

O circuito distingue o ângulo interno ϕ do ângulo físico α aplicado à rotação:

$$\boxed{\alpha_j = \frac{T_j}{2\pi}\phi_j},$$

onde T_j é o período do gate: $T_j = 4\pi$ para CY ou $T_j = 2\pi$ para CCY. Isso permite parametrizar os gates de forma consistente mesmo quando suas rotações têm convenções periódicas diferentes.

Por que o estado de Dicke é uma superposição e não uma carteira escolhida?

No sistema nCk queremos uma superposição *apenas* de bitstrings que contêm k algarismos 1. O estado simétrico de Dicke é definido por

$$\boxed{|D_k^n\rangle = \frac{1}{\sqrt{\binom{n}{k}}} \sum_{x:\,|x|=k} |x\rangle}, \qquad |x| = \sum_i x_i.$$

O símbolo $|x\rangle$ representa um bitstring em notação quântica e $|x|$ é seu número de uns. O fator $1/\sqrt{\binom{n}{k}}$ garante a normalização da soma, isto é, a probabilidade total igual a 1. Como todas as amplitudes têm módulo igual, **todas as carteiras válidas são igualmente prováveis** nesse estado ideal.

Para $4C2$, a expressão fica visível:

$$|D_2^4\rangle = \frac{1}{\sqrt{6}}\big(|0011\rangle + |0101\rangle + |0110\rangle + |1001\rangle + |1010\rangle + |1100\rangle\big).$$

Cada um dos seis bitstrings tem probabilidade $|1/\sqrt{6}|^2 = 1/6$. O propósito dos θ aprendidos é *redistribuir essas probabilidades* de acordo com a energia do Hamiltoniano, sem abandonar a cardinalidade exigida.

Este princípio e um circuito determinístico de preparação são descritos por Bärtschi e Eidenbenz, *Deterministic Preparation of Dicke States* (2019) [1]. É importante distinguir o **estado de Dicke ideal**, o **circuito que o prepara** e as **correções angulares aprendidas** sobre o mesmo circuito.

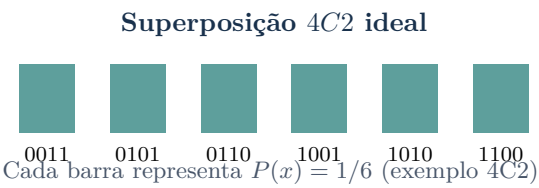


Figura 1: O estado de Dicke atribui a mesma probabilidade aos bitstrings de cardinalidade fixa; o aprendizado procura mudar essa distribuição.

Relação com trabalhos de carteiras: o uso de ansätze derivados de Dicke para otimização de carteiras já tem antecedentes relevantes em Wang *et al.* [6] e Scursulim *et al.* [7]. O ponto investigado aqui é *como gerar os ângulos a partir do Hamiltoniano usando uma rede única*, e não reivindicar a invenção da preparação de Dicke ou da otimização de carteiras com circuitos parametrizados.

Por que a porta CY pode precisar de 4π para repetir a operação?

No código, `CY` é um **bloco de rotação controlada em torno do eixo** Y (controlada-R_y), não simplesmente a porta Pauli-Y discreta. Uma rotação de um qubit é

$$R_y(\alpha) = \begin{pmatrix} \cos(\alpha/2) & -\sin(\alpha/2) \\ \sin(\alpha/2) & \cos(\alpha/2) \end{pmatrix}.$$

Observe duas identidades:

$$R_y(\alpha + 2\pi) = -R_y(\alpha), \qquad R_y(\alpha + 4\pi) = R_y(\alpha).$$

O sinal de menos parece inofensivo quando *todo* o estado recebe a mesma fase global. Mas, quando a rotação é **controlada**, somente o ramo ativado pelo controle recebe esse sinal. Assim a fase é relativa e pode afetar interferências posteriores.

Com controle c e alvo t,

$$CR_y(\alpha) = |0\rangle\langle 0|_c \otimes I_t + |1\rangle\langle 1|_c \otimes R_y(\alpha).$$

Então $CR_y(\alpha + 2\pi)$ não é geralmente a mesma operação que $CR_y(\alpha)$, porque apenas o bloco controlado muda de sinal. Já $CR_y(\alpha + 4\pi) = CR_y(\alpha)$, exatamente. **Essa é a explicação do período 4π para o parâmetro físico α no bloco CY implementado.**

Bloco do código	Rotação condicional efetiva	Período usado para o parâmetro físico
CY	$CR_y(\alpha)$	4π
CCY	$CCR_y(2\alpha)$, com dois controles	2π para α (pois 2α percorre 4π)

O nome "CCY" no projeto também é um *rótulo de implementação*, não uma única porta Pauli-Y. O circuito utiliza a decomposição com $R_y(\alpha)$, CCX e $R_y(-\alpha)$, equivalente a uma rotação duplamente controlada de ângulo 2α.

Por que a fórmula interna unifica os dois casos? A variável ϕ_j é escolhida para que

$$\alpha_j = \frac{T_j}{2\pi}\phi_j.$$

Para CY, $T_j = 4\pi$ e $\alpha_j = 2\phi_j$. Para CCY, $T_j = 2\pi$ e $\alpha_j = \phi_j$; como o bloco CCY efetua $2\alpha_j$, as duas operações implementam a rotação condicional desejada a partir da referência analítica. **O período se refere à operação completa, não simplesmente à periodicidade de** $\sin^2(\alpha/2)$ **de uma medição isolada.**

Os 18 números de cada gate, um por um, e o motivo de cada um

O Transformer não recebe simplesmente "uma carteira". Ele recebe **uma linha de 18 números para cada um dos** g **gates**. Os números descrevem a posição do gate, o tamanho do problema, a geometria da operação, informações financeiras dos ativos associados e o contexto energético das carteiras amostradas.

Na notação abaixo: j é a posição do gate, começando em zero; i e l são os índices dos qubits ligados a ele; $g = k(n-k)$ é o número de gates; $\mathbf{1}[\cdot]$ vale 1 se a condição for verdadeira e 0 caso contrário. O denominador $\max(g-1, 1)$ evita divisão por zero quando há somente um gate.

Nº	**O número que entra**	**De onde vem e por que é útil**
1	$\frac{j}{\max(g-1,1)}$	**Posição relativa do gate.** O primeiro gate tem valor 0; o último tem valor 1 se $g > 1$. Indica *em que ponto da sequência* a operação aparece. Divide-se por $g-1$ para comparar circuitos de tamanhos distintos.
2	$\frac{n}{n+12}$	**Escala do problema.** Informa quantos ativos participam. Em vez de dar simplesmente $n = 4$, usamos uma transformação limitada entre 0 e 1, evitando números de tamanho excessivo. O 12 é uma constante de engenharia escolhida na parametrização; não é uma constante física.
3	$\frac{k}{n}$	**Fração selecionada.** Distingue problemas que escolhem poucos ativos daqueles que escolhem muitos. Em $4C2$, vale 0,5; em $10C4$, 0,4. Indica quão restrita é a carteira.
4	$\frac{i}{n-1}$	**Posição do primeiro qubit.** Diz qual índice financeiro/quântico participa da operação, em escala de 0 a 1. Se $i = 1$ e $n = 4$, vale 1/3.
5	$\frac{l}{n-1}$	**Posição do segundo qubit.** Complementa o atributo 4 e permite distinguir operações associadas a pares diferentes. Se $l = 3$ e $n = 4$, vale 1.
6	$\mathbf{1}$[CY]	**Indicador de tipo.** Vale 1 quando o gate é CY e 0 caso contrário. Diz à rede qual classe de rotação está sendo aplicada, pois gates de tipos distintos não produzem necessariamente o mesmo efeito.

Nº	O número que entra	De onde vem e por que é útil
7	**1**[CCY]	**Segundo indicador de tipo.** Vale 1 quando o gate é CCY e 0 caso contrário. Nos gates considerados, um dos indicadores 6 e 7 vale 1 e o outro vale 0. Isso evita exigir que a rede descubra o tipo apenas pelos índices.
8	$\frac{\phi_{D,j}}{\pi}$	**Ângulo de referência de Dicke.** Informa qual é a configuração inicial conhecida antes de qualquer aprendizagem. Dividir por π apresenta o ângulo numa escala comparável entre operações.
9	$\frac{T_j}{4\pi}$	**Período do gate.** Identifica sua convenção angular: vale 1 para CY e 0,5 para CCY. Ajuda a relacionar a correção interna à rotação física efetiva.
10	$\sqrt{\frac{l-i}{l+1}}$	**Geometria analítica local.** Depende da distância e da posição dos qubits. É o valor q_{il} usado para construir o ângulo de Dicke. **Não é uma derivada e não requer o Jacobiano da distribuição de probabilidades.**
11	μ_i	**Retorno esperado anualizado do primeiro ativo.** Mostra à rede o incentivo financeiro associado ao índice i. Retornos maiores podem tornar desejável selecionar esse ativo, dependendo do risco conjunto.
12	μ_l	**Retorno esperado anualizado do segundo ativo.** Permite comparar os dois ativos envolvidos no gate. Sem esses valores, a posição quântica não informa se o ativo é financeiramente atraente.
13	Σ_{ii}	**Variância anualizada do primeiro ativo.** É um indicador da oscilação individual do retorno de i, necessário para entender o componente de risco da função de energia.
14	Σ_{ll}	**Variância anualizada do segundo ativo.** Cumpre a mesma função para l, permitindo comparar riscos individuais.
15	Σ_{il}	**Covariância anualizada entre os dois ativos.** Mede sua variação conjunta e adiciona informação que os dois retornos ou variâncias isolados não mostram. É uma informação *local*: a rede não recebe diretamente toda a matriz Σ.
16	$\bar{E}_c$	**Média das energias das 32 carteiras sorteadas.** Dá uma referência aproximada da escala do problema. Uma energia aparentemente baixa só faz sentido quando comparada com uma referência.
17	s_c	**Desvio-padrão das 32 energias de contexto.** Informa se as carteiras amostradas possuem energias muito parecidas ou muito dispersas. Ajuda a rede a perceber quão sensível é a escala energética.
18	$E_{c,\min}$	**Menor energia entre as 32 amostras de contexto.** Oferece uma referência de boa carteira já observada, **sem afirmar que seja o ótimo global**. Não é um rótulo de ângulos ótimos.

Vamos preencher os 18 valores do primeiro gate do 4C2

O primeiro gate usa $j = 0$, $g = 4$, $n = 4$, $k = 2$, $i = 1$ (B), $l = 3$ (D), tipo CCY. Como vimos, $\phi_D = \pi/4$, $T = 2\pi$ e $q_{1,3} = 1/\sqrt{2}$. Usando o nosso exemplo de contexto com 32 sorteios, o vetor

bruto é:

$$T_0 = \big(0,\ 0{,}25,\ 0{,}50,\ 0{,}33333,\ 1,\ 0,\ 1,\\ 0{,}25,\ 0{,}50,\ 0{,}707107,\ 0{,}08,\ 0{,}06,\\ 0{,}030,\ 0{,}020,\ 0{,}003,\ -0{,}13825,\\ 0{,}017475,\ -0{,}167\big).$$

Para deixar a ordem completamente transparente:

Posições	Valor no exemplo	Interpretação imediata
1–3	0; 0,25; 0,50	posição, n, fração escolhida
4–5	0,33333; 1	posições dos qubits B e D
6–7	0; 1	é CCY, não CY
8–10	0,25; 0,50; 0,707107	referência e geometria
11–12	0,08; 0,06	retornos de B e D
13–15	0,030; 0,020; 0,003	variâncias e covariância
16–18	−0,13825; 0,017475; −0,167	resumo das 32 energias

Os outros três gates têm seus próprios índices, tipos e ângulos de Dicke. Os atributos de contexto, porém, são *repetidos* nas quatro linhas, porque descrevem o mesmo Hamiltoniano financeiro.

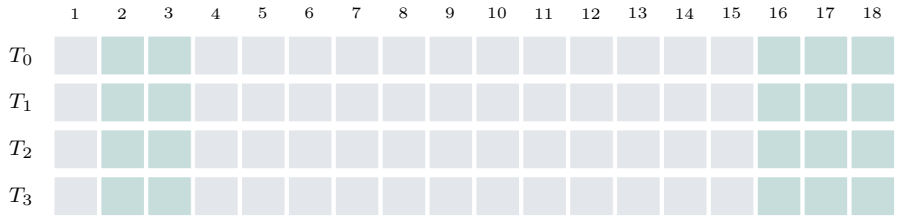
O que significa "posições dos qubits B e D"?

Os nomes **A, B, C e D** são apenas etiquetas dos **quatro ativos do exemplo**: A tem índice 0, B tem índice 1, C tem índice 2 e D tem índice 3. Assim, $i = 1$ significa que o primeiro índice envolvido nesse gate é o qubit correspondente ao ativo **B**; $l = 3$ corresponde ao ativo **D**. O mesmo par determina as colunas μ_i, μ_l e Σ_{il}.

Não confunda a letra do ativo **B** com o B usado mais adiante para contar o *número de shots*; também não confunda o ativo **D** com o símbolo $|D_k^n\rangle$ usado para o estado de Dicke. As letras cumprem papéis distintos conforme o contexto.

Quais valores são repetidos nas quatro linhas de T_H?

Não existem quatro linhas idênticas. O que temos são quatro linhas (uma por gate) e determinadas *colunas constantes ao longo dessas linhas*. Neste exemplo, as colunas **2, 3, 16, 17 e 18** se repetem *necessariamente* porque representam propriedades do Hamiltoniano completo: $n/(n+12)$, k/n, $\bar{E}_c$, s_c e $E_{c,\min}$. Outros atributos podem coincidir ocasionalmente, por exemplo quando dois gates compartilham um índice ou o mesmo tipo.



Azul-esverdeado: colunas repetidas por representarem o mesmo Hamiltoniano.

Figura 2: Mapa esquemático da entrada de $4C2$: quatro tokens diferentes, com cinco colunas necessariamente comuns.

Como muda a entrada se escolhermos 5 ou 6 ativos entre 10?

A rede pode receber quantidades *diferentes de tokens* sem mudar o número de atributos de cada token:

Escolha	Carteiras válidas	$g = k(n-k)$	Forma de T_H
$10C4$	$\binom{10}{4} = 210$	24	24×18
$10C5$	$\binom{10}{5} = 252$	25	25×18
$10C6$	$\binom{10}{6} = 210$	24	24×18

Por que $10C4$ e $10C6$ têm o mesmo número de combinações? Cada escolha de quatro ativos corresponde a um conjunto complementar de seis, e vale $\binom{n}{k} = \binom{n}{n-k}$. **Isso não significa que a energia, os gates ou os tokens sejam idênticos.**

Exemplos reais da regra de construção do *primeiro gate* em cada topologia:

Topologia	j	i	l	g	Tipo	$\phi_{D,0} = \arccos\sqrt{(l-i)/(l+1)}$
10C5	0	4	9	25	CCY	0,785398
10C6	0	3	9	24	CCY	0,684719

Em 10C5, o primeiro token começa com

$$\left(\underbrace{0}_{1}, \underbrace{10/22}_{2}, \underbrace{5/10}_{3}, \underbrace{4/9}_{4}, \underbrace{9/9}_{5}, \underbrace{0}_{6}, \underbrace{1}_{7}, \underbrace{0{,}785398/\pi}_{8}, \underbrace{1/2}_{9}, \underbrace{\sqrt{5/10}}_{10}, \dots \right),$$

seguido pelas informações financeiras $\mu_4, \mu_9, \Sigma_{44}, \Sigma_{99}, \Sigma_{49}$ e pelos três resumos do contexto. Em 10C6, o primeiro token difere já no terceiro atributo (6/10), no índice $i = 3$ e no ângulo analítico. **Não inventamos valores financeiros para esses tokens: eles serão calculados com o conjunto específico de dez ativos e sua janela histórica.**

Um token não representa uma carteira escolhida: representa *uma operação no circuito* contextualizada pelo Hamiltoniano.

Assim, a entrada completa é

$$\boxed{T_H = \begin{pmatrix} T_0 \\ T_1 \\ T_2 \\ T_3 \end{pmatrix} \in \mathbb{R}^{4\times 18}}.$$

Para $10C4$, teremos 24 linhas e 18 colunas. Não são 18 números para o problema inteiro; são **18 números por gate**.

Por que ainda padronizamos os 18 atributos?

Um índice normalizado pode variar entre 0 e 1, enquanto as energias são negativas e as covariâncias podem ser pequenas. Para evitar que uma coluna domine outra somente por sua unidade numérica, calculamos **média e desvio-padrão de cada coluna** com as instâncias de treinamento de 2024:

$$z_{jh} = \operatorname{clip}\left(\frac{T_{jh} - m_h}{\max(\sigma_h, 10^{-5})}, -10, 10 \right).$$

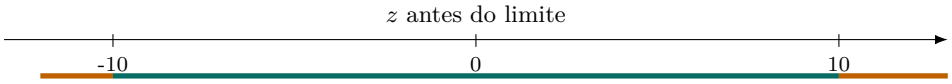
Aqui, $h = 1, \dots, 18$ identifica a coluna; m_h e σ_h são média e desvio dessa coluna no treino; clip limita valores extremos. Os números m_h, σ_h são reutilizados em validação e teste. **Não recalculamos a padronização com os dados de teste**, pois isso introduziria vazamento de informações.

O que significa `clip`?

A função $\mathrm{clip}(z, -10, 10)$ é um **limitador numérico**: se $z < -10$, devolve -10; se $-10 \le z \le 10$, devolve o próprio z; se $z > 10$, devolve 10.

$$\mathrm{clip}(z, -10, 10) = \begin{cases} -10, & z < -10, \\ z, & -10 \le z \le 10, \\ 10, & z > 10. \end{cases}$$

Exemplo: uma característica padronizada com valor $z = 2{,}3$ continua 2,3; $z = 17$ vira 10; $z = -14$ vira -10. Isso evita que uma característica extremamente atípica domine o cálculo da rede.



Valores extremos são "cortados" em -10 ou $+10$.

Cuidado: `clip` protege numericamente, mas também pode descartar informação sobre um Hamiltoniano muito diferente da base de treinamento. Por isso é necessário medir quantas entradas de teste atingem os limites -10 ou 10. O recorte não é uma operação física do circuito.

Como o Transformer transforma 18 números em um ângulo?

O Transformer não memoriza uma carteira pronta. Ele aprende uma função cujos parâmetros ajustáveis são os pesos da rede, representados por Θ.

Primeiro, cada linha de 18 números vira uma representação de 48 números

Para cada token T_j, temos

$$z_j^{(0)} = \mathrm{LayerNorm}(WT_j + b), \qquad W \in \mathbb{R}^{48\times 18}.$$

O produto $WT_j + b$ combina os 18 atributos de diferentes maneiras; LayerNorm regulariza a escala da representação dentro da rede. Agora cada gate é representado por 48 números internos, chamados *características latentes*.

Um exemplo de como 18 números viram 48, sem "inventar 48 ativos"

Uma camada linear computa cada uma das 48 saídas como uma combinação ponderada das 18 entradas:

$$z_{j,r} = \sum_{h=1}^{18} W_{rh}\, T_{j,h} + b_r, \qquad r = 1, \ldots, 48.$$

Aqui r identifica a nova característica aprendida, e h identifica um dos 18 atributos de entrada. Os pesos W_{rh} e os vieses b_r são ajustados pelo treinamento; **os 48 números não são preços, não são gates e não são 48 ângulos**.

Miniatura com três atributos e duas saídas, somente para entender a multiplicação:

$$\begin{pmatrix} 2 & 1 & 0 \\ -1 & 0 & 3 \end{pmatrix} \begin{pmatrix} 0{,}2 \\ 0{,}4 \\ 0{,}1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0{,}8 \\ 0{,}1 \end{pmatrix}.$$

A mesma ideia é aplicada a uma matriz aprendida de 48×18; em seguida entra a `LayerNorm`. A **função da expansão para 48 dimensões** é dar ao Transformer representações internas mais expressivas para combinar posição do gate, geometria e informação financeira.

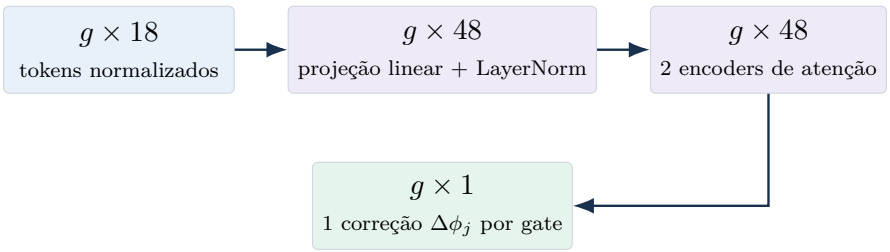


Figura 3: A dimensão 48 é o tamanho das representações latentes; a rede continua devolvendo apenas g correções angulares.

Relação com aprendizagem dos parâmetros quânticos: predição neural amortizada de ângulos não é inédita: Miao *et al.* [8] e Amosy *et al.* [9] são antecedentes. Qracle usa uma rede de grafos para representar conjuntamente Hamiltoniano e ansatz [10]; Chen *et al.* empregam um Transformer como otimizador *iterativo* de QAOA [11]. Nossa hipótese específica é combinar *um token por operação de Dicke*, correções residuais e um *único forward*; sua originalidade exata requer revisão bibliográfica sistemática, não uma afirmação absoluta de prioridade.

Depois, o mecanismo de atenção compara gates entre si

Duas camadas `TransformerEncoder` analisam todos os tokens. Cada uma usa quatro cabeças de atenção. Isso permite à representação de um gate usar informações presentes nos outros gates, em vez de olhar somente para seus próprios 18 valores.

De forma simplificada, uma cabeça de atenção calcula

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V, \qquad \text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^\mathsf{T}}{\sqrt{12}}\right)V.$$

A divisão por $\sqrt{12}$ aparece porque os 48 canais estão divididos em quatro cabeças de dimensão 12. Os pesos W_Q, W_K, W_V são aprendidos. A operação softmax transforma pontuações em pesos de combinação.

Essas relações de atenção são **aprendidas a partir dos dados**; elas não impõem de antemão a recorrência triangular ou uma alternância obrigatória de sinais entre ângulos.

Por último, sai uma correção angular para cada gate

Depois das duas camadas, cada gate possui uma representação final $z_j^{(2)}$. Uma projeção escalar produz

$$\boxed{\Delta\phi_j = 1{,}2\tanh(w_\theta^\mathsf{T} z_j^{(2)} + b_\theta)}.$$

A função tanh limita a correção ao intervalo entre $-1{,}2$ e $+1{,}2$, evitando variações sem limite na saída.

Abrindo a equação $\Delta\phi_j$ termo por termo

$$\Delta\phi_j = \underbrace{1{,}2}_{\text{escala máxima}} \underbrace{\tanh\left(\underbrace{w_\theta^\mathsf{T}}_{\text{pesos da cabeça}} \underbrace{z_j^{(2)}}_{48\text{ números do gate}} + \underbrace{b_\theta}_{\text{viés}}\right)}_{\text{limita a saída entre } -1 \text{ e } 1}.$$

- j: índice do gate, entre 0 e $g-1$;
- $z_j^{(2)} \in \mathbb{R}^{48}$: representação do gate *após* duas camadas de atenção, já informada pelos demais gates;

- $w_\theta \in \mathbb{R}^{48}$: pesos que combinam esses 48 valores em um número;
- b_θ: viés que pode deslocar esse número;
- tanh: função que limita o resultado entre -1 e $+1$;
- 1,2: limite de amplitude da **correção interna**, em unidades angulares da parametrização usada no projeto;
- $\phi_{D,j} + \Delta\phi_j$: ângulo **interno final** enviado à conversão para a rotação física α_j.

Por exemplo, se $w_\theta^\top z_j^{(2)} + b_\theta = 0{,}5$, temos $\tanh(0{,}5) \approx 0{,}4621$ e $\Delta\phi_j \approx 0{,}5545$. Assim, para um gate com $\phi_{D,j} = 0{,}7854$, resulta $\phi_j \approx 1{,}3399$. Este exemplo é só aritmético, não uma predição extraída de um checkpoint. O ângulo interno final é

$$\boxed{\phi_j = \phi_{D,j} + \Delta\phi_j}.$$

A cabeça angular começa com pesos e viés zerados; por isso, **antes de aprender qualquer coisa, a rede devolve exatamente a configuração de Dicke**. Durante o treinamento, ela aprende correções a partir da qualidade das carteiras amostradas.

Por exemplo, os quatro valores de referência do $4C2$, arredondados, são

$$\phi_D \approx (0{,}7854,\ 0{,}9553,\ 0{,}6155,\ 0{,}7854).$$

Se a rede produzisse, *hipoteticamente*,

$$\Delta\phi = (+0{,}10, -0{,}08, +0{,}20, -0{,}15),$$

teríamos

$$\boxed{\phi \approx (0{,}8854,\ 0{,}8753,\ 0{,}8155,\ 0{,}6354)}.$$

Esses números são **saída do Transformer**. Ainda não são uma lista de ativos e também não são probabilidades.

Cuidado: A arquitetura mantém uma cabeça histórica chamada HF e um adaptador, mas eles não participam efetivamente da perda nem acrescentam um gradiente de Hellmann–Feynman ao treinamento analisado. Por isso não atribuímos os resultados a essa cabeça.

Por que a arquitetura principal é encoder-only?

A arquitetura consolidada usa **duas camadas de Transformer encoder**, quatro cabeças de autoatenção e dimensão interna 48. Cada posição de gate interage com as demais pela autoatenção, sem decoder e sem um ciclo de previsão, medição e correção angular na inferência.

Importante: a expressão "previsão gate a gate" significa *uma previsão específica para cada gate*, mas *todas são produzidas no mesmo forward*. Não se executa uma otimização sequencial independente de cada gate. A formulação da autoatenção vem de Vaswani *et al.* [2].

O que os ângulos fazem dentro do circuito?

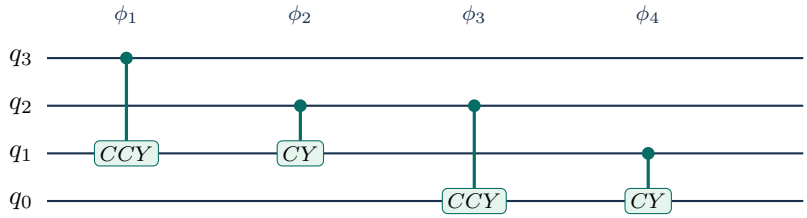
O circuito transforma o estado inicial em uma combinação quântica das carteiras possíveis com cardinalidade k:

$$|\psi(\phi)\rangle = U(\phi)|x_{\text{inicial}}\rangle = \sum_{\substack{x\in\{0,1\}^n \\ \sum_i x_i = k}} a_x(\phi)|x\rangle.$$

A amplitude $a_x(\phi)$ pode ser um número complexo. A probabilidade de medir a carteira x é

$$\boxed{P_\phi(x) = |a_x(\phi)|^2}, \qquad \sum_{x:\sum_i x_i = k} P_\phi(x) = 1.$$

Exemplo real da estrutura $4C2$: quatro tokens, quatro correções e quatro blocos



Blocos *esquemáticos*: cada CY/CCY é decomposto em CX, RY e CCX no código.
Pares (i, l) em ordem: $(1,3)$, $(1,2)$, $(0,2)$ e $(0,1)$; não representam quatro redes diferentes.

Figura 4: Os quatro ângulos são inferidos *simultaneamente*. Cada um controla um bloco específico do circuito; o resultado depende do efeito conjunto de todos os blocos.

Os ângulos controlam as amplitudes e, portanto, as probabilidades. **O circuito não escreve uma carteira única e determinística: cada medição pode devolver um bitstring diferente.**

Para ilustrar, imagine a seguinte distribuição. **Ela foi escolhida manualmente para fins didáticos; não foi calculada a partir dos ângulos hipotéticos anteriores.**

Carteira	Energia	Probabilidade ilustrativa
A+B	−0,135	10%
A+C	−0,167	40%
A+D	−0,126	5%
B+C	−0,154	20%
B+D	−0,112	5%
C+D	−0,138	20%
Total		100%

O circuito é medido várias vezes (cada medição é um *shot*). Se fizermos 20 medições, o resultado *poderia* ser: 2 vezes A+B, 7 A+C, 1 A+D, 5 B+C, 1 B+D e 4 C+D. São contagens ilustrativas, não uma garantia de que a amostra seguirá exatamente as probabilidades da tabela.

A energia de cada bitstring medido é calculada pela mesma função $E_H(x)$. Selecionamos a menor energia *entre os bitstrings que realmente apareceram*. No exemplo, A+C apareceu e o melhor valor observado foi −0,167.

O que acontece se nenhuma medição encontrar a melhor carteira?

O resultado é a melhor carteira *amostrada*, não necessariamente a melhor carteira que existe. Isso é essencial: uma distribuição pode favorecer carteiras razoáveis e ainda assim deixar de medir a ótima dentro do orçamento disponível.

Na avaliação usual, adotamos $B = 128$ shots. Durante as estimativas SPSA do treino completo, cada lado da perturbação usa $S = 96$ shots. São quantidades distintas porque treinamento e avaliação têm papéis diferentes.

Onde o Monte Carlo entra, literalmente, e por que ele não é a rede

Monte Carlo (MC) significa estimar quantidades por *amostragem aleatória*. Ele é empregado aqui em três situações distintas:

1. **Antes da rede:** sorteamos 32 carteiras válidas para construir o contexto energético.
2. **Durante o treino:** os *shots* amostram bitstrings com probabilidades $P_\phi(x)$ definidas pelo circuito, permitindo calcular uma recompensa sem examinar todos os estados. Os dois lados SPSA usam 96 shots cada.
3. **Na auditoria matemática:** sorteamos repetidamente lotes de B medições para comparar o mínimo observado com a esperança calculada analiticamente; essa auditoria *não* modifica os pesos.

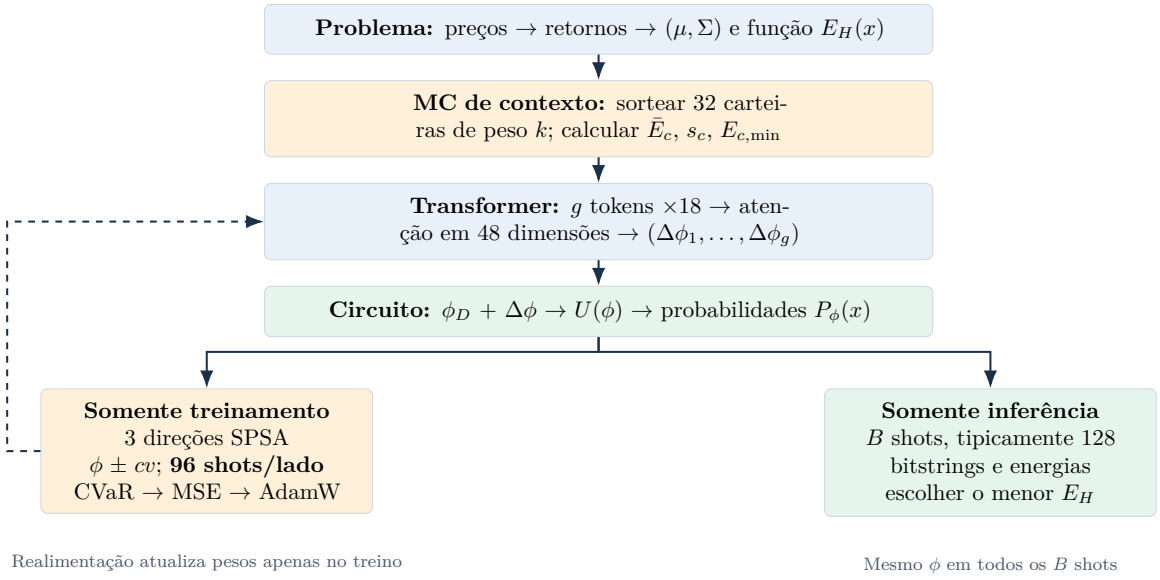


Figura 5: Os dois lugares essenciais de Monte Carlo: antes da rede, para resumir as energias de contexto; depois do circuito, para estimar CVaR no treinamento ou amostrar carteiras na inferência. No treino, cada direção SPSA exige duas configurações perturbadas.

MC **não substitui o Transformer**. A rede prevê $\Delta\phi$; o MC produz amostras da distribuição gerada pelo circuito. O MC **pode prejudicar a qualidade das atualizações indiretamente** se usarmos tão poucos shots que a estimativa do CVaR fique muito ruidosa. Por outro lado, permite treinar sem enumerar $\binom{n}{k}$ estados. O efeito deve ser medido em experimentos pareados de orçamento, e não presumido.

Cuidado: O experimento usa simulação clássica do circuito (por exemplo, Aer/MPS) com medições amostradas, e não resultados de um processador quântico físico. A simulação tem seu próprio custo, que pode aumentar com o emaranhamento.

Como a rede aprende se ninguém lhe entrega os ângulos corretos?

A peça central do método é que podemos avaliar uma *carteira produzida* mesmo sem saber a carteira ótima. Cada lote de medições fornece um sinal de qualidade.

Por que não usar somente a melhor medição?

Uma primeira ideia é usar

$$E_{\text{best}} = \min(E_1, \ldots, E_S).$$

Mas, se mudarmos um pouco os ângulos e obtivermos exatamente a mesma melhor energia, esse número não detectará a diferença entre as duas distribuições. O sinal fica frequentemente nulo.

Outra ideia seria utilizar a média de todas as energias:

$$\overline{E}_{\text{shots}} = \frac{1}{S}\sum_{s=1}^{S} E_s.$$

Ela muda com mais frequência, mas pode favorecer concentração excessiva em algumas carteiras que não são as melhores para nosso objetivo final.

Por isso usamos o CVaR de 10% das melhores energias

Ordenamos as energias medidas da menor para a maior:

$$E_{(1)} \leq E_{(2)} \leq \cdots \leq E_{(S)}.$$

Escolhemos as $m = \lceil 0{,}10S \rceil$ menores e calculamos a média:

$$\boxed{C_{0,10} = \frac{1}{m}\sum_{a=1}^{m} E_{(a)}}.$$

Quando $S = 96$, temos $m = \lceil 9{,}6 \rceil = 10$. Assim, a recompensa considera as dez melhores energias observadas, em vez de apenas uma.

CVaR 10% com uma amostra minúscula, número por número

Considere apenas dez energias *fictícias* para facilitar a ordenação:

$$(-0{,}20, -0{,}13, -0{,}11, -0{,}18, -0{,}09, -0{,}16, -0{,}14, -0{,}12, -0{,}10, -0{,}15).$$

Depois de ordenar da menor para a maior:

$$(-0{,}20,\ -0{,}18,\ -0{,}16,\ -0{,}15,\ -0{,}14,\ -0{,}13,\ -0{,}12,\ -0{,}11,\ -0{,}10,\ -0{,}09).$$

Com 10 shots, 10% significa $m = 1$ observação e $C_{0,10} = -0{,}20$; com 20 shots, seriam as *duas* menores; com 96, são as **dez menores**. Note que repetições de um mesmo bitstring contam como medições separadas: se o mesmo bitstring excelente aparece cinco vezes, sua energia também pode aparecer cinco vezes no cálculo do CVaR.

O trabalho de Barkoutsos *et al.*, *Improving Variational Quantum Optimization using CVaR*, demonstra a motivação para usar a cauda das menores energias em otimização quântica variacional [3]. Nosso uso está adaptado ao treinamento de uma rede que prevê diretamente os ângulos.

No exemplo hipotético de 20 shots, $m = \lceil 2 \rceil = 2$. Como A+C apareceu sete vezes e é a melhor energia, as duas menores observações são $-0{,}167$ e $-0{,}167$:

$$C_{0,10} = \frac{-0{,}167 - 0{,}167}{2} = -0{,}167.$$

Em uma amostra com mais energias distintas entre as melhores, o CVaR refletiria também essas outras observações.

Cuidado: Aqui chamamos de CVaR a *cauda inferior da energia*, porque minimizar energia é desejável. Em outras áreas, CVaR muitas vezes é definido sobre a cauda superior de perdas. A convenção deve ser declarada para evitar confusão.

Para comparar problemas de escalas diferentes, a função objetivo usada no treinamento é normalizada:

$$\boxed{L_0(\phi; H) = \frac{C_{0,10}(\phi; H) - \bar{E}_c}{s_H}}, \qquad s_H = \max(s_c, 0{,}02).$$

Subtrair $\bar{E}_c$ centraliza a pontuação em relação ao contexto. Dividir por s_H evita que a escala de um Hamiltoniano domine os demais. O piso 0,02 impede que uma dispersão muito pequena produza números enormes; é uma escolha numérica, não uma constante universal.

Função objetivo, termo por termo, com números de exemplo

$$L_0(\phi; H) = \frac{\underbrace{C_{0,10}(\phi; H)}_{\text{média das 10 melhores energias entre 96}} - \underbrace{\bar{E}_c}_{\text{média das 32 carteiras de contexto}}}{\underbrace{\max(s_c, 0{,}02)}_{\text{escala mínima segura}}}.$$

ϕ são os ângulos que queremos ajustar; H identifica o problema financeiro. Se, por exemplo, $C_{0,10} = -0{,}19$, $\bar{E}_c = -0{,}14$ e $s_c = 0{,}03$, então

$$L_0 = \frac{-0{,}19 - (-0{,}14)}{0{,}03} = -1{,}6667.$$

Se outra configuração produzisse $C_{0,10} = -0{,}17$ para o *mesmo* H, daria $-1{,}0$: como minimizamos, $-1{,}6667$ é melhor. **O valor absoluto da função não é uma energia física: é um escore normalizado.**

Direita e esquerda dos ângulos: como surge a direção de aprendizado?

Suponha que a rede produziu os g ângulos ϕ. Queremos descobrir uma direção na qual a perda tende a diminuir. Não calculamos o Jacobiano completo das probabilidades dos bitstrings. Em vez disso, usamos diferenças de avaliações do circuito.

Escolhemos uma direção aleatória com entradas $+1$ e -1:

$$v \in \{-1, +1\}^g.$$

Formamos dois conjuntos de ângulos:

$$\phi^+ = \phi + cv, \qquad \phi^- = \phi - cv, \qquad c = 0{,}18.$$

Isto quer dizer que, em uma direção, alguns ângulos aumentam e outros diminuem; na outra direção, fazemos exatamente o inverso. Executamos o circuito para cada lado e avaliamos L_0 a partir das energias medidas.

Se uma direção está associada a perda menor, ela pode ser útil para corrigir os ângulos. A estimativa SPSA do gradiente da perda é

$$\boxed{\widehat{g}_L = \frac{1}{3}\sum_{d=1}^{3} \frac{\widehat{L}(\phi + cv_d) - \widehat{L}(\phi - cv_d)}{2c}\, v_d}.$$

A barra em $\widehat{L}$ lembra que a perda foi *estimada por shots*, portanto contém ruído. Usamos três direções aleatórias v_1, v_2, v_3, em vez de uma só.

SPSA: o que cada símbolo da derivada quer dizer?

$$\widehat{g}_L = \underbrace{\frac{1}{3}}_{\text{média de três tentativas}} \sum_{d=1}^{3} \underbrace{\frac{\overbrace{\widehat{L}(\phi + cv_d)}^{\text{perda após deslocar para um lado}} - \overbrace{\widehat{L}(\phi - cv_d)}^{\text{perda no lado contrário}}}{2\underbrace{c}_{0{,}18}}}_{\text{inclinação aproximada nessa direção}} \underbrace{v_d}_{\text{vetor de sinais } \pm 1}.$$

- $\widehat{g}_L \in \mathbb{R}^g$: estimativa conjunta da direção de aumento da perda;
- v_d: vetor que decide quais ângulos aumentam e quais diminuem simultaneamente;
- c: magnitude da perturbação em cada coordenada;
- $\widehat{L}(\phi \pm cv_d)$: perdas medidas usando *dois circuitos completos* diferentes;
- $2c$: distância utilizada na aproximação por diferenças centrais;
- $d = 1, 2, 3$: repetição com três direções diferentes, para reduzir a dependência de uma só direção sorteada.

Exemplo com um único parâmetro: se $c = 0{,}1$, $\widehat{L}(\phi + 0{,}1) = -1{,}2$ e $\widehat{L}(\phi - 0{,}1) = -1{,}0$, então a derivada estimada é

$$\frac{-1{,}2 - (-1{,}0)}{0{,}2} = -1.$$

O sinal negativo indica que, localmente, *aumentar* esse ângulo favoreceu a redução da perda. No caso real, o SPSA testa **todos os ângulos simultaneamente**, portanto cada resultado contém efeitos conjuntos dos gates.

Este procedimento é a aproximação por perturbações simultâneas de Spall [4], e sua vantagem é não exigir duas avaliações separadas para cada um dos g parâmetros em toda atualização.

Quando $\widehat{g}_L \neq 0$, a direção normalizada de **descida** é

$$d = -\frac{\widehat{g}_L}{\|\widehat{g}_L\|}.$$

O sinal de menos aparece porque queremos *minimizar* a perda. Com $\eta = 0{,}06$, construímos um pequeno deslocamento desejado:

$$\boxed{\phi_{\text{alvo}} = \phi + \eta d}.$$

Se a estimativa é nula, não obtemos uma direção útil nessa atualização.

O custo de uma atualização completa é

$$3 \text{ direções} \times 2 \text{ lados} \times 96 \text{ shots} = \boxed{576 \text{ shots}}.$$

Com 35 épocas e 48 atualizações por época, temos 1.680 atualizações e $1.680 \times 576 = \boxed{967.680}$ shots de treinamento por rede, sem contar a validação.

Onde está, afinal, a função de perda do próprio Transformer?

A rede produz uma correção $\Delta\phi_\Theta$. Como desejamos que ela siga a direção de descida estimada, criamos um alvo local

$$\Delta\phi_{\text{alvo}} = \text{stopgrad}(\Delta\phi_\Theta) + \eta d.$$

O operador stopgrad significa: usamos o valor calculado, mas não deixamos a diferenciação automática voltar através dele. A perda que atualiza os pesos internos do Transformer é

$$\boxed{\mathcal{L}_\Theta = \frac{1}{g}\sum_{j=1}^{g}(\Delta\phi_{\Theta,j} - \Delta\phi_{\text{alvo},j})^2}.$$

O otimizador AdamW modifica os pesos Θ da rede para que suas próximas correções se movam na direção desejada.

A MSE de atualização dos pesos, explicada símbolo por símbolo

$$\mathcal{L}_\Theta = \frac{1}{g}\sum_{j=1}^{g}\Big[\underbrace{\Delta\phi_{\Theta,j}}_{\text{saída atual da rede}} - \underbrace{\Delta\phi_{\text{alvo},j}}_{\text{saída atual desligada + correção SPSA}}\Big]^2.$$

Θ são **todos os pesos treináveis do Transformer**, e não os θ_j dos gates. O erro é elevado ao quadrado para que alterações em qualquer direção tenham magnitude positiva e desvios maiores contem mais. O fator $1/g$ produz uma média sobre os gates, permitindo comparar circuitos com 24, 25 ou 36 parâmetros.

O alvo *não* é um $\theta^\star$ obtido por VQE; ele é um **passo local de aprendizado**. Como $\Delta\phi_{\text{alvo}} = \text{stopgrad}(\Delta\phi_\Theta) + \eta d$, com $\eta = 0{,}06$, esta MSE pode apresentar valor quase constante e ainda assim gerar gradientes para os pesos da rede.

AdamW: quem aprende o quê?

Depois de calcular $\nabla_\Theta\mathcal{L}_\Theta$ por retropropagação, o **AdamW** ajusta os pesos Θ da rede. Ele *não* move diretamente um θ_j por uma regra fixa permanente: altera o mapa aprendido que, no próximo Hamiltoniano, produzirá novos θ_j.

Em forma esquemática, o AdamW acompanha uma média móvel do gradiente (m_t), uma média móvel do seu quadrado (v_t), e realiza uma atualização adaptativa com decaimento de pesos separado:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2,$$

$$\Theta_{t+1} = \Theta_t - \eta_{\text{Adam}}\frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}+\varepsilon} - \eta_{\text{Adam}}\lambda_w\Theta_t.$$

Aqui $g_t = \nabla_\Theta\mathcal{L}_\Theta$; β_1, β_2 controlam a memória dos gradientes; $\widehat{m}_t, \widehat{v}_t$ corrigem o viés inicial dessas médias; η_{Adam} é a taxa de aprendizado; ε previne divisão por zero; e λ_w é o decaimento de pesos, aplicado separadamente. O código emprega `torch.optim.AdamW`; sua formulação é discutida por Loshchilov e Hutter [5].

O que diferencia nosso treino de um VQE iterativo?

Durante o treinamento, usamos perturbações SPSA e medições para *ensinar os pesos da rede*. Na inferência, para um Hamiltoniano novo, **não se executa SPSA, nem AdamW, nem a busca iterativa dos ângulos**; o encoder já treinado fornece toda a configuração angular em um único forward. Trabalhos anteriores estabeleceram a possibilidade de previsão neural de parâmetros variacionais, como Miao *et al.* [8] e Amosy *et al.* [9].

Por que "saída imediata" e "saída da busca completa" são duas coisas?

Etapa	O que sai?	Por que ainda precisamos da próxima etapa?
Transformer, um forward	$\phi_1, \ldots, \phi_g$, os g ângulos finais	Ângulos descrevem como o circuito deve operar, mas ainda não são uma seleção de ativos.
Circuito parametrizado	Amplitudes $a_x(\phi)$ e probabilidades $P_\phi(x)$	Uma distribuição oferece várias carteiras possíveis; precisamos obter amostras.
Medições (shots)	Lista de bitstrings $x^{(1)}, \ldots, x^{(B)}$	Cada bitstring precisa ser avaliado usando os retornos e riscos do Hamiltoniano.
Busca completa	x_{best} e $E_H(x_{\text{best}})$	É a melhor carteira efetivamente observada no orçamento B, não necessariamente o ótimo global.

Assim, *um forward* significa somente uma execução da rede para gerar todos os ângulos. A busca completa ainda faz medições do circuito. **Não há um novo VQE ajustando o vetor de ângulos na inferência.**

Uma única previsão e diferentes quantidades de medições

Depois de um forward, a rede produziu *uma* configuração ϕ. Podemos usar a **mesma configuração** em $B = 1, 8, 32$ ou 128 medições. O número B altera a chance de encontrar a carteira ótima, mas **não altera os ângulos inferidos**.

Se o bitstring ótimo tiver probabilidade $p_\star$ na distribuição produzida pelo circuito, cada medição independente tem probabilidade $1 - p_\star$ de *não* encontrá-lo. Em B tentativas independentes:

$$P(\text{pelo menos um acerto}) = 1 - (1 - p_\star)^B.$$

O termo $1 - p_\star$ corresponde a uma falha; $(1 - p_\star)^B$ corresponde a falhar em todas as medições; o termo externo $1-$ transforma esse valor na probabilidade de ao menos um sucesso.

Exemplo ilustrativo: com $p_\star = 0{,}25$, um shot acerta com 25% de probabilidade; oito shots acertam ao menos uma vez com $1-0{,}75^8 \approx 89{,}99\%$. **Nenhum dos oito shots executa novamente o Transformer.**

Nos casos pequenos, a distribuição exata obtida por vetor de estado permite auditar essa fórmula sem ruído estatístico de amostragem. Nos grandes, o resultado precisa ser estimado por medições: a taxa de acerto do ótimo só pode ser declarada se o ótimo clássico tiver sido *certificado* independentemente. A previsão one-forward não implica solução ótima garantida nem vantagem quântica.

Resultados consolidados: o que já foi demonstrado e o que ainda falta

As métricas abaixo pertencem a experimentos diferentes, com Hamiltonianos e procedimentos definidos em seus próprios notebooks. **Não devem ser interpretadas como uma comparação direta entre linhas**. Os valores representam resultados auditados, não expectativas de uma próxima execução.

Experimento	Evidência e limite da conclusão
G5.38: mesmo encoder, dados completos de 2024	Cinco seeds e 9.672 instâncias visitadas por seed; nos painéis de 2025 e 2026, melhora da energia frente ao ponto inicial Dicke. **Não** houve vitória estrita sobre o 1-SWAP em 128 avaliações no painel apresentado.
G5.38: caso real 10C5	O encoder atribuiu 54,1752% de probabilidade ao bitstring ótimo `0011001110`, com energia −1,784789. Isso mede qualidade da distribuição *naquela instância*, não em todas.
G5.39: auditoria exata com poucos shots	A análise por vetor de estado encontrou probabilidade média do ótimo de aproximadamente 24,61% em um shot para o encoder de referência, contra 0,38% para Dicke, nos casos avaliados. O desempenho muda quando aumenta o orçamento de medições.
Limite de interpretação	Os testes de 2025–2026 são retrospectivos, já consultados durante o desenvolvimento; o computador quântico foi *simulado*, não executado em hardware real. Não há vantagem quântica demonstrada.

O que é sólido: um encoder com pesos congelados produz correções de ângulo por gate em um forward; o circuito mantém a cardinalidade; em alguns experimentos a distribuição resultante concentra muito mais massa no ótimo do que Dicke. **O que continua em aberto:** robustez em espaços de busca combinatoriamente difíceis, desempenho perante heurísticas clássicas fortes e custos físicos de medição e circuito.

O que isso significa: Na apresentação, use o $4C2$ como ilustração do funcionamento e o resultado $10C5$ como exemplo de evidência experimental. Não use o exemplo numérico inventado de $4C2$ como uma medida de precisão do modelo.

O caminho completo, agora em uma única sequência

A ideia inteira pode ser lida nesta ordem:

1. **Observar preços**: calcular o retorno diário de cada ativo.
2. **Resumir os dados**: estimar μ, o retorno médio anualizado, e Σ, a covariância anualizada.
3. **Definir o que é uma boa carteira**: escolher exatamente k ativos e avaliar cada carteira observada com $E_H(x) = \frac{1}{2}x^\mathsf{T}\Sigma x - \mu^\mathsf{T}x$.
4. **Criar uma escala local**: amostrar 32 carteiras válidas e calcular $\bar{E}_c$, s_c e $E_{c,\min}$.
5. **Descrever cada gate**: montar exatamente 18 informações por operação; haverá $g = k(n-k)$ linhas.
6. **Produzir os ângulos**: o Transformer transforma os tokens em correções $\Delta\phi$ e soma essas correções aos ângulos de Dicke.
7. **Produzir carteiras**: o circuito usa os ângulos para determinar probabilidades e as medições retornam bitstrings com exatamente k uns.
8. **Avaliar as medições**: calcular a energia de cada carteira produzida, sem necessidade de conhecer previamente a ótima.
9. **Aprender**: durante o treinamento, usar CVaR, SPSA e AdamW para modificar os pesos do Transformer; em inferência, fazer apenas um forward e medir o circuito.
10. **Verificar se a aprendizagem compensa**: comparar com a configuração de Dicke, amostragem uniforme e 1-SWAP, usando o mesmo orçamento de candidatos e painéis de teste independentes sempre que possível.

A saída imediata da rede é, portanto,

$$\boxed{\text{dados financeiros e estrutura dos gates} \xrightarrow{\text{Transformer}} \phi_1, \ldots, \phi_g}.$$

A saída da busca completa é

$$\boxed{\phi \xrightarrow{\text{circuito e shots}} \{x^{(1)}, \ldots, x^{(B)}\} \xrightarrow{E_H} \left(x_{\text{best}}, E_{\text{best}}\right)}.$$

A distinção mais importante é esta: o Transformer produz *ângulos*; o circuito transforma os ângulos em uma distribuição de *carteiras*; a função de energia permite decidir qual carteira observada é melhor. As funções de perda servem para ensinar à rede quais alterações angulares tendem a melhorar esse processo, mas cada nova perda precisa provar sua utilidade em testes controlados.

Referências e o papel de cada uma no método

1. **Bärtschi, A.; Eidenbenz, S.** *Deterministic Preparation of Dicke States* (2019). `https://arxiv.org/abs/1904.07358`. Estado de Dicke e preparação determinística.
2. **Vaswani, A. et al.** *Attention Is All You Need* (2017). `https://arxiv.org/abs/1706.03762`. Encoder, embeddings e atenção.
3. **Barkoutsos, P. K. et al.** *Improving Variational Quantum Optimization using CVaR. Quantum* 4, 256 (2020). `https://doi.org/10.22331/q-2020-04-20-256`. Objetivo variacional CVaR.
4. **Spall, J. C.** *Multivariate Stochastic Approximation Using a Simultaneous Perturbation Gradient Approximation* (1992). `https://doi.org/10.1109/9.119632`. SPSA.
5. **Loshchilov, I.; Hutter, F.** *Decoupled Weight Decay Regularization* (2019). `https://arxiv.org/abs/1711.05101`. AdamW.
6. **Wang, S. et al.** *Variational quantum eigensolver with linear depth problem-inspired ansatz for solving portfolio optimization in finance* (2024). `https://arxiv.org/abs/2403.04296`. Carteiras, Dicke e CVaR em VQE.
7. **Scursulim, P. et al.** *Multiclass portfolio optimization via variational quantum Eigensolver with Dicke state ansatz* (2026). `https://pmc.ncbi.nlm.nih.gov/articles/PMC12905263/`. Aplicação de Dicke parametrizado em carteiras financeiras.
8. **Miao, J.; Hsieh, C.-Y.; Zhang, S.-X.** *Neural network encoded variational quantum algorithms* (2024). `https://arxiv.org/abs/2308.01068`. Mapeamento neural de parâmetros de Hamiltonianos para circuitos variacionais.
9. **Amosy, O. et al.** *Iteration-Free quantum approximate optimization algorithm using neural networks. Quantum Machine Intelligence* 6, 38 (2024). `https://doi.org/10.1007/s42484-024-00159-y`. Predição neural de parâmetros com inferência sem loop variacional por instância.
10. **Zhang, C.; Jiang, L.; Chen, F.** *Qracle: A Graph-Neural-Network-based Parameter Initializer for Variational Quantum Eigensolvers* (2025). `https://arxiv.org/abs/2505.01236`. Uso conjunto de informação do Hamiltoniano e estrutura do ansatz.
11. **Chen, K.-C. et al.** *Transformers as Intrinsic Optimizers for Quantum Approximate Optimization Algorithm* (*preprint*, 2026). `https://arxiv.org/abs/2609.19392`. Transformer como otimizador iterativo no QAOA, distinto da nossa inferência angular direta.

O que isso significa: O método consolidado é: Dicke parametrizado, tokens financeiros/geométricos por gate, encoder-only, um forward para todos os ângulos, medições do circuito e aprendizagem com CVaR/SPSA/AdamW. Cada referência fundamenta uma peça; a combinação e sua vantagem ainda exigem comparação experimental e revisão bibliográfica cuidadosa.